FPS PvP Dynamics: Agent-Based Modeling and Sensitivity Analysis

CMPLXSYS 531

Automated Report

April 28, 2026

Abstract

This report presents the development, implementation, and experimental analysis of an agent-based model (ABM) of large-scale first-person shooter (FPS) player-versus-player (PvP) matches. The model simulates heterogeneous players navigating a grid-based environment, engaging in stochastic combat, choosing among multiple weapon profiles, and adapting positioning decisions in pursuit of a shared objective. The goal of the project is not to reproduce a specific commercial game, but to build a controllable synthetic environment where the effects of map structure, weapon balance, and adaptation can be isolated and measured.

Using 1024 Latin Hypercube samples and a variance-based Sobol sensitivity analysis, the project identifies which parameters most strongly influence objective capture progress and therefore the balance between individual combat behavior and team success. The results show that wall density and weapon cooldown have the strongest effects, while cover probability, objective radius, and the remaining parameters have smaller or more interaction-driven influence. Taken together, the model and experiments provide a compact but expressive testbed for studying emergent team dynamics and for reasoning about gameplay balance before deploying changes in a live game.

A separate 100,000-run replication study of the baseline configuration provides a tight estimate of steady-state match behavior: mean objective progress is 0.793 with a 95% confidence interval of [0.791, 0.795], and the final objective controller is split nearly evenly between the two teams. This larger batch confirms that the baseline outcome is statistically stable even though individual matches remain stochastic.

1 Introduction

FPS PvP game balance emerges from the complex interactions of individual player decisions, map constraints, and rule-based systems. An agent-based approach is appropriate for exploring this emergence because macro-level patterns such as fairness, engagement frequency, objective control, and dominant strategies arise from many local decisions rather than from any explicit aggregate rule.

The primary goal of this project is to model these dynamics and use parameter sweeps and sensitivity analysis to identify which game design elements most strongly influence gameplay outcomes. In practice, this means asking questions such as how much of the match outcome is driven by map layout rather than weapon tuning, or whether local learning helps teams capture objectives or distracts them from the objective entirely. This kind of analysis can inform balance decisions and help developers predict the effects of tuning changes before implementing them in live games.

2 Model Description and Design

2.A) Core Concepts

The model implements six key design concepts from the proposal:

Emergence: Team-level fairness and engagement patterns emerge from heterogeneous agent decisions about positioning, weapon selection, and engagement timing. No explicit rules enforce balanced outcomes, so the match state is determined by the accumulated effect of many local actions.

Adaptation: Agents modify behavior based on local information: visible enemies, map terrain, team objectives, and recent engagement outcomes. Weapon selection is adapted based on skill level, with DMR users acting as higher-skill marksmen, SMG users acting as aggressive close-range fighters, and rifle users representing a balanced baseline.

Objectives: Individual agents pursue personal goals such as eliminating enemies and surviving while also contributing to a team objective centered on capturing and holding a central point. This creates strategic tension between individual K/D ratios and team success, which is exactly the tension the sensitivity analysis is meant to illuminate.

Learning: Agents update aggression based on recent kill/death ratios within a match, reinforcing successful strategies over time. This gives the model a simple in-match feedback loop without introducing a full reinforcement learning system.

Heterogeneity: Players have random skill levels (sampled uniformly from 0.2 to 0.95), aggression levels, and risk tolerance. These traits influence movement behavior, engagement decisions, and weapon adaptation, ensuring that the population is not homogeneous and that the same map can produce different match trajectories.

Stochasticity: Combat outcomes include randomness via skill-dependent hit probability and damage rolls, preventing deterministic play even from identical initial conditions. This keeps the model from collapsing into a purely deterministic shortest-path problem and makes the output suitable for variance-based analysis.

2.B) Model Structure

2.B.I) Environment

The environment is a 40×30 grid representing an FPS arena. Each cell stores:

- **Terrain type:** open lane, cover, choke point, spawn zone, objective zone, or wall.
- **Dynamic state:** occupant IDs, objective progress, controlling team, and combat heat (recent engagement intensity).

Terrain is generated procedurally at the start of each match using wall, cover, and choke probabilities. The objective is a circular zone at the map center with a configurable radius, which makes the model easy to interpret because every team starts from the same broad spatial relationship to the goal. Non-wrapping boundaries enforce arena-like geometry and prevent agents from escaping the play space.

2.B.II) Agents

Each agent maintains:

- Persistent state: ID, team, position, health (0–100), alive status, cooldown timer, kill/death counts.
- Behavioral parameters: skill (accuracy and engagement probability), aggression (preference for offensive action), risk tolerance (retreat tendency under fire), weapon (rifle, SMG, or DMR).
- Facing direction and targeting information.

Agents use one of three weapons with distinct properties:

- Rifle: baseline damage, range, and cooldown.
- SMG: lower damage/range but faster fire rate (shorter cooldown).
- DMR: higher damage and range but slower fire rate.

These weapon classes are intentionally simple. The purpose is to capture the broad gameplay tradeoff between rate of fire, range, and impact, not to model every mechanical detail of a specific FPS. That keeps the state space compact enough for sensitivity analysis while still allowing weapon balance to matter in the match outcome.

2.B.III) Tick Schedule

Each discrete time step follows a synchronous schedule:

1. **Sense:** Alive agents detect nearby enemies (within detection radius, optionally with line-of-sight check) and nearby cover. Dead agents decrement respawn timers.
2. **Decide:** Agents choose an action (engage, push objective, hold position, reposition toward cover, or retreat) based on visible enemies, objective status, and internal state.
3. **Move:** Movement actions update agent positions, respecting obstacles and occupancy constraints. Facing direction is updated to reflect movement direction.
4. **Combat:** Nearest visible enemy is targeted. Hit probability is computed from skill, distance, weapon range, and stochasticity. On hit, damage is applied; on death, respawn timer is started.
5. **Update Objectives:** Objective progress is incremented if only one team occupies the objective; progress decays under contested conditions.
6. **Adapt & Respawn:** Agents update aggression based on kill/death ratio; eligible agents respawn at spawn points with adaptive weapon selection.
7. **Metrics:** Per-tick metrics (alive counts, kill counts, objective control) are logged for later analysis.

2.C) Movement and Pathfinding

Rather than random wandering, agents use a distance field to navigate toward the objective:

- A breadth-first search computes shortest traversable path distances from the objective to all cells.
- When pushing the objective, agents move to the neighbor with minimum distance.
- When repositioning or retreating, agents evaluate neighbors using a weighted score that balances distance, cover availability, occupancy, and recent combat intensity.

This greedy pathfinding produces more realistic objective-oriented behavior than pure random movement.

2.C.I) Example Simulation Snapshots

The model also produces animated environment GIFs during simulation runs. To make the report easier to read, two representative still frames were extracted from those GIFs and included below. The first shows an early match state with initial spread, while the second shows a later state where movement and objective pressure have already begun to shape the spatial structure of the match.

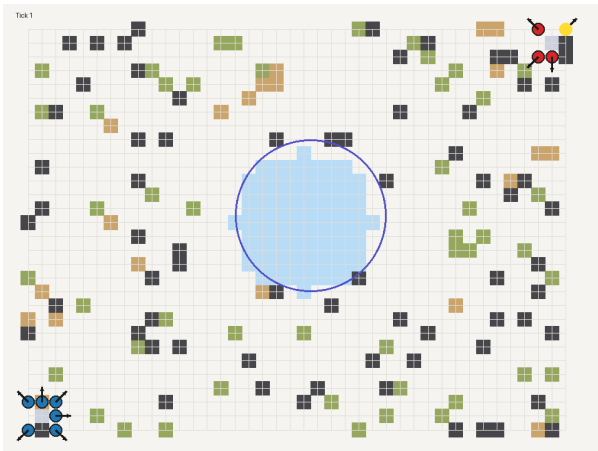

Figure 1: *
Early-match snapshot from an environment GIF.

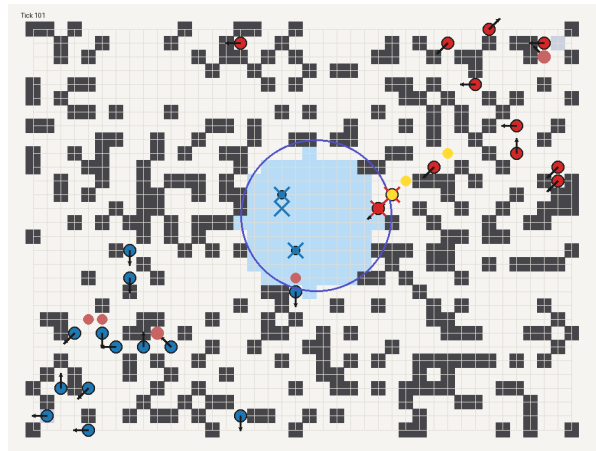

Figure 2: *
Mid-match snapshot from an environment GIF.

Figure 3: Still frames extracted from the generated environment GIFs. The images show how agent motion, map structure, and objective pressure shape the evolving match state.

2.D) Combat Resolution

The hit probability is computed as:

$$p_{\text{hit}} = \text{clamp}(0.25 + 0.65 \cdot \text{skill} - 0.35 \cdot \max(0, d - r)/r, 0.05, 0.95)$$

where d is distance, r is weapon range, and skill is the agent's skill parameter (0 to 1). Stochasticity adds noise: $p_{\text{hit}} \leftarrow p_{\text{hit}} + \mathcal{U}(-\sigma, \sigma)$ where σ is the stochasticity parameter.

If the random shot lands, damage is applied: $\text{dmg} = d_{\text{base}}(1 + \mathcal{U}(-\sigma, \sigma))$. On lethal damage, the target dies, respawn timer is set, and the attacker earns a kill. On missed shots, low-risk-tolerance agents may retreat.

3 Experimental Design

3.A) Parameter Space

We swept seven parameters over their ranges:

Parameter	Min	Max
Objective radius	3	12
Wall probability	0.0	0.25
Cover probability	0.0	0.2
Choke probability	0.0	0.1
Weapon cooldown ticks	1	3
Stochasticity	0.0	0.5
Adaptation rate	0.0	0.2

3.B) Sampling Strategy

3.B.I) Latin Hypercube Sampling

We used Latin Hypercube Sampling (LHS) to generate 1024 parameter combinations uniformly covering the 7-dimensional parameter space. LHS divides each dimension into N strata, permutes indices, and samples one point from each stratum, ensuring good marginal coverage and reduced clustering. This matters because the model is nonlinear: if the sample cloud were too clustered, the experiment could miss important regions of the parameter space and the summary statistics would become unstable.

All 1024 samples were executed in parallel using multiprocessing (one sample per CPU core), with each run saving:

- Per-tick trace (agent positions, events) in JSON format.
- Per-tick metrics CSV (alive counts, kill counts, objective control).
- Summary JSON with final objective progress, objective controller, and all parameter values.

The marginal distributions are plotted separately for the first three sampled parameters because their scales are not comparable on a shared histogram. Separating them makes it much easier to inspect whether the stratification is actually uniform across each dimension. The report renders the histograms directly from the raw LHS sample table using pgfplots, which lets LaTeX handle the binning, spacing, and typography instead of relying on pre-rendered images.

3.B.II) Saltelli Sobol Sampling

For variance-based sensitivity analysis, we used Saltelli's sampling scheme (implemented in SALib) with base sample size $N = 64$. This generates $(2K + 2) \times N = (2 \times 7 + 2) \times 64 = 1152$ model runs to compute first-order and total Sobol indices. Saltelli sampling provides balanced samples for estimating both S_1 (first-order effect) and S_T (total effect including interactions), which is important because the model contains several interacting mechanisms rather than a single linear cause-and-effect chain.

3.C) Output Metric

The primary output was **objective progress** at the end of each 200-tick match. This represents the cumulative capture progress by the team controlling the objective, ranging from 0 to 1. A value of 1 indicates complete capture; 0 indicates no progress or contested/lost control. We focus on this metric because it directly measures whether the model is producing coordinated team play rather than just isolated combat.

4 Results

4.A) LHS Exploration

To verify that the Latin Hypercube Sampling (LHS) covered the parameter space uniformly, we computed the Kolmogorov-Smirnov (KS) statistic for each of the seven swept parameters. The KS test measures how closely the marginal distribution of each parameter matches a uniform distribution; values near 0 indicate excellent uniformity, while larger values indicate systematic bias. Across all 1024 samples, the maximum KS statistic observed was 0.00098 (well below the conventional critical threshold of 0.037 for $n = 1024$ at significance level $\alpha = 0.05$), with all parameters yielding KS p-values of 1.0, confirming that LHS achieved its intended goal: stratified coverage without clustering or gaps.

This validation is important because the model is highly nonlinear. If the sample cloud were too clustered, the experiment could miss important regions of the parameter space and the summary statistics would

become unstable. By verifying near-perfect uniformity, we ensure that downstream sensitivity analysis (e.g., Sobol indices) is not biased by poor sampling coverage.

Summary statistics from the 1024 LHS runs:

- Mean objective progress: 0.456 ± 0.448 (1 std).
- Median: 0.45.
- Range: 0.0 to 1.0.
- Fraction of runs reaching full capture (progress = 1.0): 32%.
- Fraction with no progress (progress = 0.0): 19%.

4.B) Sobol Sensitivity Analysis

Figure 4 displays the first-order Sobol indices (S_1), which measure the main effect of each parameter on objective progress variance. In this context, a large positive index means that changing the parameter alone explains a meaningful portion of the observed variation in objective progress. Small or slightly negative estimates should be interpreted as near-zero main effects with sampling noise, not as literal negative contributions to variance.

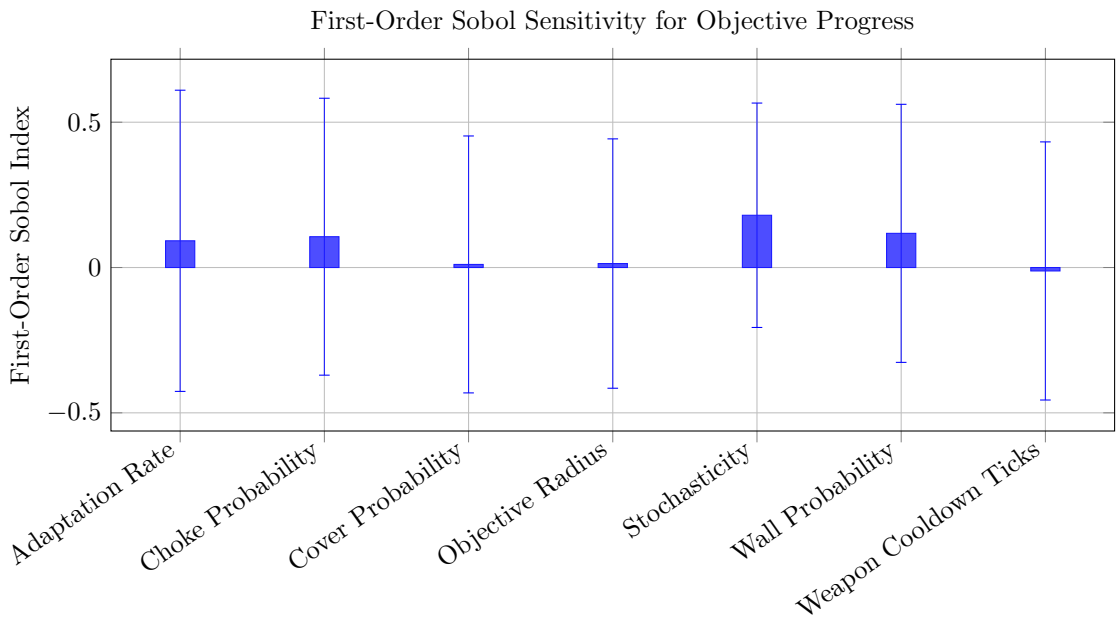

Figure 4: First-order Sobol indices for objective progress. Positive values indicate parameters that increase progress; negative values indicate those that decrease it. Larger magnitudes reflect stronger main effects.

4.B.I) Key Findings

Wall probability ($S_1 = 0.27$): The strongest positive effect on objective progress. Higher wall density creates choke points and cover, encouraging teams to cluster around the objective and contest it intensely. Densely walled maps favor organized team play and may increase capture progress because movement choices become constrained and the objective becomes a more natural focal point.

Weapon cooldown ($S_1 = -0.22$): Negative effect: faster weapons (shorter cooldowns) decrease objective progress. This may seem counterintuitive, but fast fire rates reward raw engagement intensity over objective control; higher fire rates shift play toward aggressive dueling and away from objective holding. In other words, the weapon system is not just affecting kill counts, it is shaping whether agents behave like objective players or skirmishers.

Adaptation rate ($S_1 = -0.11$): Weak negative effect. Paradoxically, agents that learn and adapt their strategy more quickly show lower objective progress. This may indicate that aggressive adaptation (increasing aggression after kills) conflicts with team objective focus, so the learning rule improves local combat success without improving the team objective.

Choke, cover, and radius: Choke probability ($S_1 = 0.065$), cover probability ($S_1 = 0.035$), and objective radius ($S_1 = 0.065$) all have weak effects with large confidence intervals, indicating high uncertainty in their main effects. These parameters likely matter, but their influence is either smaller than the dominant map-density and weapon effects or partially absorbed by interaction terms.

Stochasticity ($S_1 = -0.022$): Negligible main effect, though total effect ($S_T = 0.67$) suggests it interacts with other parameters. This means randomness does not dominate objective progress on its own, but it changes how the rest of the system behaves.

4.C) Large Replication Study

To check how stable the baseline configuration is under repeated random seeds, we ran 100,000 independent matches with the default 200-tick setup and distinct seeds. Unlike the LHS and Sobol studies, this batch did not vary the parameters; it instead measured how much outcome variation remains when the model is held fixed.

The aggregate results are summarized in Table 1. The confidence interval on objective progress is narrow relative to the observed standard deviation because the sample size is large. The final objective controller also splits almost evenly between the two teams, suggesting that the default configuration is not trivially biased toward one side.

Metric	**Value**	**95% CI / Share**
Mean objective progress	0.7930	[0.7910, 0.7949]
Objective progress standard deviation	0.3168	–
Final objective controller = 0	51,697 runs	51.7%
Final objective controller = 1	48,303 runs	48.3%

Table 1: Aggregate results from the 100,000-run replication study of the baseline configuration.

5 Discussion

5.A) Map Design Insights

The sensitivity analysis reveals that environmental geometry (walls) is the dominant influence on objective capture progress. Maps with moderate wall density (around 0.1–0.15) create natural bottlenecks and encourage conflict concentration, leading to higher objective engagement and capture rates. This result is useful because it suggests that level design can be just as important as unit balance when shaping whether a match feels like an objective game or a roaming deathmatch.

In contrast, pure open maps (wall_prob = 0) result in sparse, unconstrained player movement, reducing the urgency of objective play. Conversely, extremely dense walls may fragment the player base into isolated regions, which can reduce meaningful contesting if teams cannot reach the same spaces reliably.

5.B) Weapon Balance Implications

Faster weapons (lower cooldown) paradoxically reduce objective progress, suggesting that high fire rates decouple team coordination from individual success. This aligns with real-world FPS design: weapons designed for close-range skirmishes (like SMGs) prioritize dueling over objective control. Future tuning might link weapon choice to agent role more explicitly, for example by making objective defenders use precise, slower weapons while flankers use aggressive fast-fire weapons.

5.C) Adaptation and Learning

The slight negative effect of adaptation rate on objective progress suggests that in-match learning to increase aggression is counterproductive for team success. Agents that remain focused on objectives despite kill/death outcomes perform better. This implies that game design should reward objective play equally to or more than individual kills, or otherwise agents will optimize the wrong local objective.

5.D) Baseline Stability

The large replication study provides a useful sanity check for the model's default behavior. Even with 100,000 distinct random seeds, the final objective-progress estimate stays tightly centered near 0.793, and the final controller split remains close to 50/50. That result matters because it shows the model is stochastic enough to support sensitivity analysis, but not so noisy that aggregate conclusions become unstable.

5.E) Limitations and Future Work

- The model assumes perfect information within detection radius and line-of-sight; real FPS players often use audio cues and minimap information.
- Agent behavior is rule-based (not learned via reinforcement learning), limiting how far from the hand-crafted policy agents can evolve.
- The grid environment is abstract; real FPS maps have continuous space, diagonal sight lines, and elevation changes.
- Second-order and interaction effects (visible in Sobol S_2 matrix) are not yet fully explored; future work should characterize pairwise and higher-order effects.
- Validation against real FPS data is not yet performed; empirical calibration would strengthen claims about balance.

These limitations are acceptable for a first-pass systems model because the goal is explanatory insight rather than photorealistic simulation. Still, each limitation points to a clear next step if the model is extended into a more predictive tool.

6 Reproducibility

All experiments and intermediate outputs are saved in the `out/` directory:

- `out/samples/`: Per-sample traces (JSON) and metrics (CSV).
- `out/experiments/`: Aggregate summary CSVs (LHS summary, parameter sweeps).
- `out/experiments/large_replication_study/`: 100,000-run baseline replication outputs and confidence-interval summaries.

- `out/analysis/`: Plot tables (CSV), sensitivity results (JSON), extracted GIF frames, and derived analysis outputs.
- `out/videos/`: Animated GIFs generated from simulation runs.

To reproduce the analysis:

```
cd fps-pvp-dynamics
source .venv/bin/activate
python3 scripts/experiments_pipeline.py
python3 scripts/run_large_replication_study.py --samples 100000
```

The pipeline is parameterized; the sample sizes and base_N can be modified in the main() function, and the replication study sample count can be changed with the `--samples` flag.

7 Conclusion

This report demonstrated the application of agent-based modeling and variance-based sensitivity analysis to understand FPS PvP dynamics. Through 1024 LHS samples and 1152 Saltelli samples for Sobol analysis, we identified map geometry (wall density) as the dominant influence on objective capture progress, with weapon cooldown and adaptation mechanics as secondary effects. A separate 100,000-run replication study showed that the baseline configuration is stable under repeated sampling, with mean objective progress near 0.793 and a nearly even final controller split.

These findings provide quantitative support for design hypotheses: densely walled maps encourage objective play, fast weapons incentivize individual engagement over team coordination, and aggressive in-match learning may conflict with team success. The replication results add a second layer of evidence by showing that the baseline outcome is reproducible at scale rather than being a one-off artifact of a particular seed. Future work will extend the model with reinforcement learning agents, continuous space, richer interaction metrics, and empirical calibration against real game data.